National Taipei University of Technology

Object-Oriented Analysis and Design

Homework #4

arch-checker

A Fluent Architectural Style Compliance Tool for Java

Team Member:

113598009 資工碩二 李俊威

Table of Contents

1. Requirement Document 3
    1.1 Change History 3
    1.2 Problem Statement 3
    1.3 Summary of System Features 3
    1.4 Use Case Diagram 4
    1.5 Use Cases 5
    1.6 Non-Functional Requirements and Constraints 10
    1.7 Glossary 10
    1.8 Software Environments 11
2. Domain Model 11
    2.1 Domain Class Diagram (Concepts) 11
    2.2 Associations 13
    2.3 Attributes 14
3. Design 15
    3.1 Logical Architecture 15
    3.2 Use-Case Realizations with GRASP Patterns 18
    3.3 Design Class Model 28
4. Measurement 30

1. Requirement Document

1.1 Change History

Version	Date	Author	Description
1.0	2026-03-03	李俊威	Initial submission (HW1) — Problem Statement
2.0	2026-03-12		Add System Features, Use Cases, NFR, Glossary (HW2)
3.0	2026-04-01		Add Domain Model; revise HW2: condense Problem Statement, rename SF-XX to FEA-XX, remove CLI commands from Use Cases, generalize error messages, remove implementation details
4.0	2026-04-17		Add Design (Logical Architecture, Use-Case Realizations with GRASP, Design Class Model); revise HW3: remove non-system-state items from UC-02/UC-03 preconditions, reformat Domain Model descriptions as tables
5.0	2026-04-22		Revise HW4: compact UC-01/UC-04 Sequence Diagrams, redraw Design Class Diagram, add Class Specifications table

1.2 Problem Statement

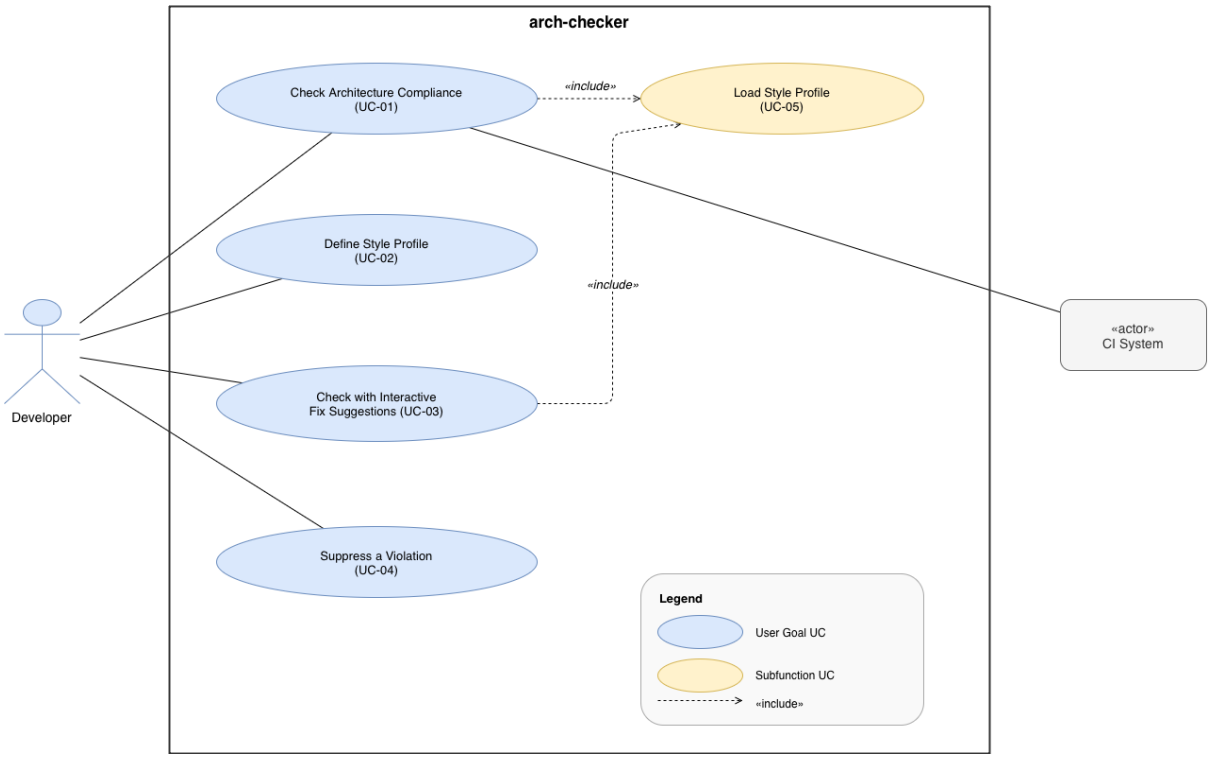
隨著軟體系統規模擴大與 AI 輔助程式碼生成的普及，人工驗證架構風格合規性日益困難，成為開發週期的瓶頸。現有工具（如 ArchUnit）雖可將架構限制寫成測試規則，但使用門檻較高，開發者需熟悉底層 API 並撰寫大量樣板程式碼，缺乏以風格為中心的高階抽象。arch-checker 是一套專為 Java 開發者設計的命令列工具，用於自動驗證 Java 專案的架構風格合規性。使用者可透過宣告式 API 以 YAML 定義架構風格設定檔（Style Profile），將命名、依賴、繼承等規則封裝為可重複使用的風格單元。系統透過 Java 原始碼的 AST 解析，精確偵測結構性違規，並產出含檔案路徑、行號與錯誤描述的完整報告，適用於本地開發、程式碼審查與持續整合情境。

1.3 Summary of System Features

ID	Feature Name	Priority	Description
FEA-01	Architecture Compliance Checking	High	掃描 Java 專案目錄，依使用者定義的 Style Profile 偵測違規
FEA-02	Style Profile Definition via YAML	High	使用者以 YAML 格式定義任意架構風格規則（DependencyRule、NamingRule、SupertypeRule、PackageRule）；工具不預

ID	Feature Name	Priority	Description
			設任何內建風格
FEA-03	Violation Report Generation	High	輸出含檔案路徑、行號、規則 ID、說明的違規報告
FEA-04	Multiple Report Formats	Low	支援 Console 與 JSON 兩種輸出格式
FEA-05	Violation Suppression	Medium	標記特定違規為「已知且接受」，下次不再回報

1.4 Use Case Diagram



Actors:

Developer (Primary)：使用工具定義架構規則並檢查 Java 專案的開發者

CI System (Secondary)：在 CI/CD pipeline 中自動觸發合規檢查，依賴 exit code 判斷結果

UC	Name	Level	Format	Triggered by
UC-01	Check Architecture Compliance	User Goal	Fully Dressed	Developer / CI System
UC-02	Define Style Profile	User Goal	Fully Dressed	Developer
UC-03	Check with Interactive Fix Suggestions	User Goal	Fully Dressed	Developer

UC	Name	Level	Format	Triggered by
UC-04	Suppress a Violation	User Goal	Brief	Developer
UC-05	Load Style Profile	Subfunction	Brief	UC-01 / UC-03 (include)

1.5 Use Cases

UC-01: Check Architecture Compliance (Fully Dressed)

Use Case	UC-01
Use Case Name	Check Architecture Compliance
Scope	arch-checker CLI
Level	User Goal
Primary Actor	Developer
Stakeholders and Interests	Developer：想要取得準確且詳細的違規報告，包含檔案路徑與行號，以便快速找出不符合規範的類別，避免費時的人工審查。 Team Lead / Architect：想確保整個程式碼庫符合團隊定義的架構規則，零違規代表架構一致性獲得維護。 CI System（次要）：想要可靠的 POSIX exit code（0/1/2）來判斷 pipeline 是否通過，不需要解讀報告內文。
Preconditions	1. arch-checker 已安裝並可在系統中存取 2. 目標 Java 專案目錄存在且可讀取 3. Style Profile YAML 檔案存在且語法正確
Success Guarantee	系統產生完整的 ViolationReport，包含：已檢查檔案數、違規清單、suppressedCount；並回傳 POSIX exit code（0 = pass，1 = violations found）
Main Success Scenario	1. Developer 發出對指定專案目錄進行架構合規檢查的要求，並指定要使用的 Style Profile 2. 系統確認專案目錄存在 3. 系統讀取並解析指定的 Style Profile YAML 檔案 4. 系統遞迴掃描專案目錄下所有 .java 檔案 5. 系統以 JavaParser 將每個 .java 檔解析為 AST（CompilationUnit） 6. 系統對每個 CompilationUnit 套用 Profile 中所有 ArchitectureConstraint 規則 7. 系統收集所有 Violation，每筆包含：檔案路徑、行號、規

	則 ID、說明文字 8. 系統過濾已被 suppress 的 Violation 9. 系統以 Console 格式輸出 ViolationReport（含違規清單與 summary） 10. 系統回傳 exit code：無違規則 0，有違規則 1
Extensions	2a. 專案目錄不存在：系統輸出錯誤訊息，exit code 2，終止 3a. Style Profile 檔案不存在：系統輸出錯誤訊息，exit code 2，終止 3b. YAML 語法錯誤：系統輸出 YAML 解析錯誤與行號，exit code 2，終止 3c. YAML 語法正確但規則定義無效（如未知的 rule type）：系統輸出 validation 錯誤清單，exit code 2，終止 4a. 專案目錄下無 .java 檔：系統輸出警告訊息，exit code 0，終止 5a. 某 .java 檔語法錯誤無法解析：系統記錄警告訊息，跳過該檔案，繼續處理其餘檔案 8a. 所有 Violation 皆已被 suppress：報告顯示全部通過（含已抑制數量），exit code 0
Special Requirements	NFR-01（Performance）：掃描 1,000 個 .java 檔案必須在 10 秒內完成（一般開發機） NFR-02（Usability）：所有錯誤訊息必須包含：錯誤原因、位置（檔案/行號）、建議修正動作 NFR-05（CI Compatibility）：exit code 遵循 POSIX 標準，可直接整合 GitHub Actions / Jenkins
Technology and Data Variations List	*a. Style Profile 接受任意 YAML 檔案路徑，不支援內建名稱 *b. 輸出格式可切換為 JSON，預設為 Console 文字格式 4a. Java 檔案掃描採遞迴深度優先走訪
Frequency of Occurrence	CI 環境：每次 push 觸發，每日可達數十次；本地開發：每小時數次
Miscellaneous	是否需支援 multi-module Maven 專案（多個 src/main/java 子目錄）？ 大型專案（>10,000 個 .java 檔）是否需要增量掃描機制？

UC-02: Define Style Profile (Fully Dressed)

Use Case	UC-02
Use Case Name	Define Style Profile
Scope	arch-checker CLI

Level	User Goal
Primary Actor	Developer
Stakeholders and Interests	Developer：想用最少工夫定義出可重複使用的架構規則，希望工具提供模板降低手寫 YAML 的門檻，並在儲存前即時得知格式是否正確。 Team Lead：想要團隊共享一份統一的 Profile 並版本控制於 Git，確保規則描述清楚且長期可維護。
Preconditions	arch-checker 已安裝並可在系統中存取
Success Guarantee	一個語法正確、至少含 1 條規則的 .yaml Profile 檔案存在於磁碟，且可被 UC-01 成功載入使用
Main Success Scenario	1. Developer 發出建立新 Style Profile 模板的要求，並指定輸出位置 2. 系統產生含預設結構與欄位說明註解的空白 YAML 模板，寫入指定位置 3. Developer 完成 YAML 編輯後，發出驗證 Style Profile 的要求 4. 系統讀取並解析 YAML 檔 5. 系統驗證每條規則的 type（NamingRule / DependencyRule / SupertypeRule / PackageRule）與所有必填欄位 6. 系統驗證每個 package pattern 的語法（只允許英數字、*、.） 7. 系統輸出驗證成功訊息，顯示載入的規則數量 8. Developer 在執行 UC-01 時指定此 Profile
Extensions	3a. Developer 跳過模板建立步驟，直接對已存在的 YAML 發出驗證要求：從 Step 4 開始，正常執行 4a. 指定檔案不存在：系統輸出錯誤訊息，終止 5a. YAML 頂層結構錯誤（不是合法的 YAML mapping）：系統輸出錯誤訊息與錯誤行號，終止 5b. 遇到未知的 rule type：系統輸出錯誤訊息，列出允許的規則類型，終止 6a. package pattern 含非法字元：系統輸出錯誤訊息，終止 7a. YAML 格式正確但 rules 清單為空：系統輸出警告訊息，表示 Profile 有效但不會產生違規，exit code 0
Special Requirements	NFR-02（Usability）：所有 validation 錯誤訊息必須清楚指出 YAML 錯誤行號，並提供修正建議 NFR-04（Extensibility）：擴充新的規則類型不應影響現有規則的合規檢查行為
Technology and	*a. Profile 格式為 YAML（以 SnakeYAML 解析），採 UTF-8

Data Variations List	編碼 *b. Developer 可使用任意文字編輯器（VS Code、IntelliJ、Vim 等），不需 IDE plugin 2a. 若輸出目錄不存在，系統自動建立
Frequency of Occurrence	每個新專案通常執行一次；規則有異動時重複執行 validate；相對低頻，以週或月為單位
Miscellaneous	是否提供 JSON Schema 供 IntelliJ / VS Code 在編輯 Profile YAML 時自動補全？ Profile 是否支援繼承（extends: <other-yaml-path>）以在既有規則上擴充？

UC-03: Check Architecture Compliance with Interactive Fix Suggestions (Fully Dressed)

Use Case	UC-03
Use Case Name	Check Architecture Compliance with Interactive Fix Suggestions
Scope	arch-checker CLI
Level	User Goal
Primary Actor	Developer
Stakeholders and Interests	Developer：想逐條審視每個 Violation 並當場決定是否 suppress，希望系統提供可操作的修復建議，而不只是列出錯誤清單。 Team Lead：想知道哪些 Violation 是「已知且接受」的技術債，並有持久化且可稽核的 suppression 紀錄，而非無聲地被略過。
Preconditions	1. 同 UC-01 的前置條件 2. Terminal 支援互動式輸入（非 pipe 重定向的 non-TTY 模式）
Success Guarantee	對每條 Violation，三種結果之一成立：(a) Suppression 記錄寫入 .arch-checker-suppress.yaml，(b) Violation 保留在報告中，(c) 流程中止；最終 ComplianceReport 反映所有決策結果
Main Success Scenario	1. Developer 發出以互動模式對指定專案進行架構合規檢查的要求，並指定 Style Profile 2. 系統完成掃描流程（同 UC-01 Step 2–7）：走訪檔案、解析 AST、套用規則、收集所有 Violation 3. 系統展示第一條 Violation，顯示：檔案路徑與行號、違反的 ArchitectureConstraint ID 與說明、RefactoringSuggestion（若存在）

	4. 系統提示是否接受 suppression? 5. Developer 輸入 y：系統將此 Violation 的 constraint ID 與時間戳記寫入 .arch-checker-suppress.yaml 6. 系統展示下一條 Violation（回到 Step 3） 7. 所有 Violation 處理完畢後，系統輸出最終 ComplianceReport（已扣除接受的 suppression） 8. 系統回傳 exit code（0 = 無未接受 Violation，1 = 有未接受 Violation）
Extensions	3a. 此 Violation 有 RefactoringSuggestion：系統在顯示 [y/n/q] 提示前，額外顯示建議操作 3b. 掃描後無 Violation：系統直接輸出全部通過訊息，不進入互動迴圈，exit code 0 5a. Developer 輸入 n：此 Violation 保留在最終報告中，系統繼續下一條（Step 6） 5b. Developer 輸入 q：系統中止互動迴圈，輸出已處理部分的 partial report，exit code 1 5c. Developer 輸入非法值（非 y/n/q）：系統輸出 Invalid input. Please enter y, n, or q:，重新等待輸入 8a. 所有 Violation 均被 suppress（全部輸入 y）：報告顯示全部通過（含已抑制數量），exit code 0
Special Requirements	NFR-02（Usability）：互動提示須清晰易懂，Developer 無需查閱文件即能理解 [y/n/q] 各選項的含義 NFR-06（Testability）：互動流程的合規性檢查邏輯必須可透過 unit test 獨立驗證，不依賴完整的 CLI 環境
Technology and Data Variations List	*a. Suppression 記錄寫入 .arch-checker-suppress.yaml，包含：constraint ID、reason（預設為 suppressed interactively）、timestamp 4a. [y/n/q] 輸入大小寫不敏感（Y、Yes 均視為接受） *b. 若 stdout 被 pipe 重定向（non-TTY），系統自動降級為非互動批次模式並輸出警告
Frequency of Occurrence	本地開發初次建立 suppression 清單時使用，通常每個新專案執行一次；後續違規出現時按需使用，整體低頻
Miscellaneous	是否需要提供 --accept-all 選項，一次 suppress 所有 Violation？ Suppression 是否應設有效期限，避免長期堆積技術債？

UC-04: Suppress a Violation (Brief)

Developer 發出抑制特定違規的要求，並提供 constraint ID 與原因說明。系統將 suppression 記錄（constraint ID、reason、timestamp）寫入 .arch-checker-suppress.yaml。

下次執行 UC-01 時，於 Step 8 過濾生效。

UC-05: Load Style Profile (Brief — Subfunction)

UC-01 與 UC-03 的 <<include>> 子流程。系統讀取使用者指定的 Style Profile YAML 檔，解析並驗證規則定義。成功後將 Profile 載入記憶體，供各 constraint checker 使用。

1.6 Non-Functional Requirements and Constraints

ID	Category	Requirement
NFR-01	Performance	完整的 compliance check（UC-01）掃描 1,000 個 .java 檔案必須在 10 秒內完成（一般開發機）
NFR-02	Usability	所有錯誤訊息必須包含：錯誤原因、位置（檔案路徑 / 行號）、建議修正動作
NFR-03	Portability	工具需在 JDK 17+ 環境（Windows / macOS / Linux）上運行，不依賴特定 IDE
NFR-04	Extensibility	系統應支援在不修改既有合規檢查流程的前提下，擴充對新架構規則類型的支援
NFR-05	CI Compatibility	exit code 遵循 POSIX 標準（0 = success，非 0 = failure），可直接整合 GitHub Actions 與 Jenkins
NFR-06	Testability	每個架構規則的合規性檢查邏輯必須可透過 unit test 獨立驗證，不依賴完整的 CLI 環境

1.7 Glossary

Term	Definition
Style Profile	使用者以 YAML 格式定義的架構規則集合，描述特定專案的架構約束；工具本身不提供任何內建 Profile
ArchitectureConstraint	Style Profile 中的單條約束規則，為 NamingRule、DependencyRule、SupertypeRule、PackageRule 的抽象父類別
SupertypeRule	一種 ArchitectureConstraint，要求特定 package 內的 class 必須 implement 指定 interface 或 extend 指定 class；以 simple name 比對，不進行 fully qualified name resolution
Violation	一筆架構違規記錄，包含：檔案路徑、行號、規則 ID、說明文字
ViolationReport	單次 check 執行的完整輸出，彙整所有 Violation 與 summary
AST (Abstract Syntax Tree)	Java 原始碼的抽象語法樹表示，由 JavaParser 解析產生，供各 ConstraintChecker 進行分析

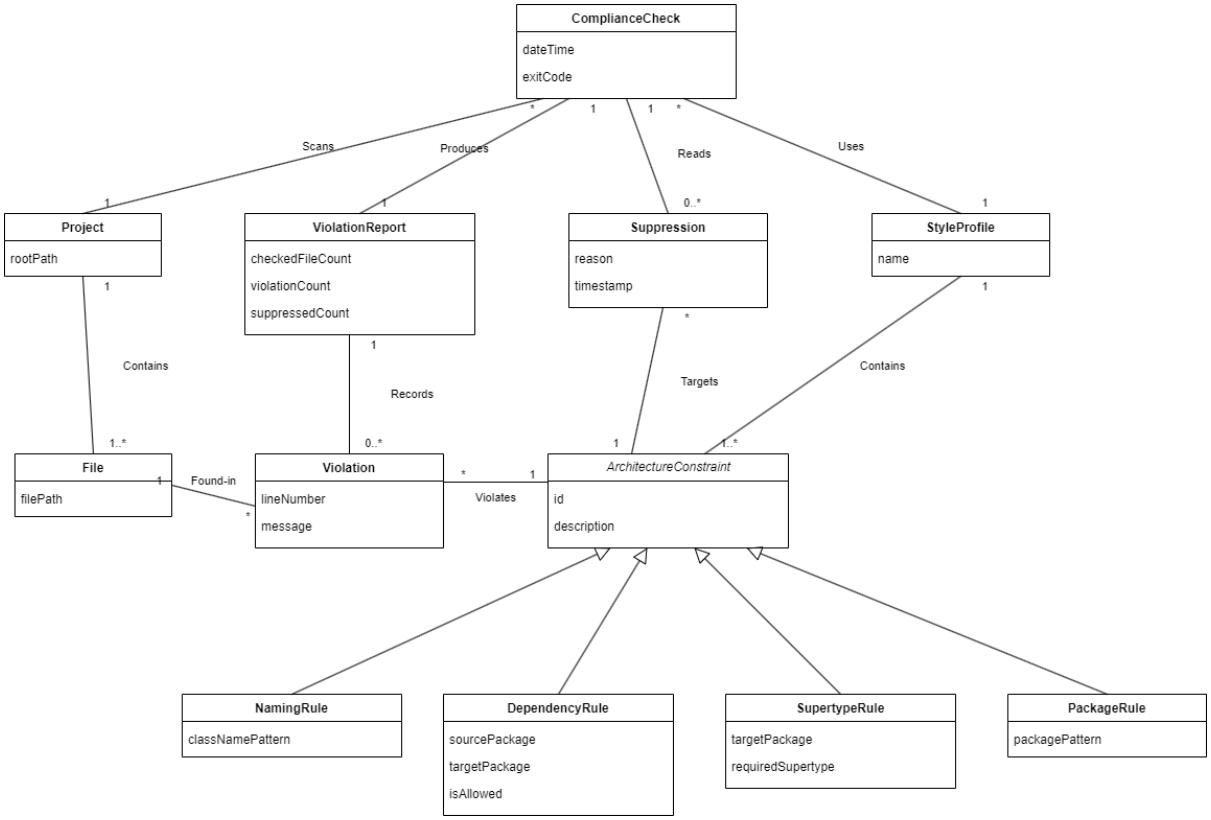
Term	Definition
CompilationUnit	JavaParser 解析單一 .java 檔案後產生的根節點物件
Suppression	開發者明確標記「此 Violation 已知且接受」的機制，使其在後續報告中不再出現
exit code	CLI 程式結束時回傳給 shell 的整數碼（0 = pass，1 = violations found，2 = error）
Package Pattern	規則定義中用來比對 Java package 名稱的萬用字元字串（如 *.ui.*）；只允許英數字、*、.

1.8 Software Environments

Item	Choice	Rationale
Development Language	Java 17 (JDK 17+)	LTS 版本穩定
AST Parser	JavaParser 3.x	純 Java，API 直覺，無需 JDK tools
CLI Framework	picocli	輕量，annotation-driven，與 Java 17 相容
Profile Format	YAML (SnakeYAML)	可讀性高，易於編輯
Test Framework	JUnit 5	unit testing
Build Tool	Maven	依賴管理方便

2. Domain Model

2.1 Domain Class Diagram (Concepts)



上圖為 arch-checker 的 Domain Class Diagram。識別出的概念類別如下：

已採納的概念類別與理由：

Concept	Rationale for Inclusion
ComplianceCheck	代表一次完整的架構合規檢查作業，對應 UC-01/UC-03 中 Developer 對專案發出檢查要求的動作。系統需記錄每次檢查的時間與結果。
StyleProfile	使用者以 YAML 定義的架構規則集合，包含多條 ArchitectureConstraint。系統在 UC-01 載入 Profile 後，於 UC-02 逐條驗證其中的約束規則。
ArchitectureConstraint	StyleProfile 中的單條約束規則（抽象概念），為 Profile 的組成元素。設為抽象類別，因其具有四個具體子類型，各自對應不同的架構約束面向。
NamingRule / DependencyRule / SupertypeRule / PackageRule	ArchitectureConstraint 的四種具體子類型，分別檢查類別命名慣例、套件間依賴方向、繼承關係要求與套件結構規範。UC-02 驗證流程中依規則類型分派不同的檢查邏輯。
Project	被檢查的目標 Java 專案，UC-01 中 Developer 指定的掃描對象。系統需知道專案路徑以定位其中的 File。
File	專案中的 .java 原始碼檔案，為 Project 的組成元素。每個 Violation 需關聯到特定的 File 以定位違規位置。
ViolationReport	單次檢查的完整報告，為 ComplianceCheck 的產出物。彙

Concept	Rationale for Inclusion
	整所有 Violation 並提供統計摘要，供 Developer 或 CI 系統判讀結果。
Violation	單筆架構違規記錄，記載哪個 File 違反了哪條 ArchitectureConstraint。為 ViolationReport 的明細項目。
Suppression	開發者標記「已知且接受」的違規抑制記錄，對應 UC-04 的功能。讓團隊可以在不修改程式碼的情況下，將特定違規排除於報告之外。

排除的候選概念與理由：

Excluded Candidate	Reason for Exclusion
CompilationUnit / AST	這是 JavaParser 函式庫的軟體產物（software artifact），屬於實作層面的概念，不應出現在 Domain Model 中。
ProfileLoader / ProfileValidator	這些是軟體類別（software classes），負責解析與驗證 Profile 的內部行為，非領域中的真實概念。
ConsoleReport / JSONReport	Console 與 JSON 是 ViolationReport 的輸出格式（SF-04），屬於呈現方式而非領域概念，不需獨立建模。
Developer / CISystem	這些是外部 Actor，不屬於系統內部的領域概念。
ExitCode	exit code 是數值型態，適合作為 ComplianceCheck 的屬性而非獨立的概念類別。

2.2 Associations

關聯的建立依據「需要記憶的關係」原則（need-to-remember associations）。下表列出各關聯的名稱、對應數量與建立理由：

Association	Cardinality	Rationale
ComplianceCheck Uses StyleProfile	* — 1	每次檢查使用一份 Profile；同一份 Profile 可被多次檢查使用。
ComplianceCheck Scans Project	* — 1	每次檢查掃描一個專案；同一專案可被多次檢查。
ComplianceCheck Produces ViolationReport	1 — 1	每次檢查產出一份報告，一對一關係。
ComplianceCheck Reads Suppression	1 — 0..*	檢查時讀取已存在的 Suppression 記錄來過濾結果。
StyleProfile Contains ArchitectureConstraint	1 — 1..*	一份 Profile 至少包含一條規則，為組合關係。

Association	Cardinality	Rationale
Project Contains File	1 — 1..*	專案包含一至多個 Java 檔案，為組合關係。
ViolationReport Records Violation	1 — 0..*	報告記錄零至多筆違規，為報告的明細項目。
Violation Violates ArchitectureConstraint	* — 1	每筆違規對應一條被違反的規則。
Violation Found-in File	* — 1	每筆違規發生在特定的 Java 檔案中。
Suppression Targets ArchitectureConstraint	* — 1	每筆抑制記錄對應一條特定規則。

此外，NamingRule、DependencyRule、SupertypeRule、PackageRule 為 ArchitectureConstraint 的泛化（generalization）子類型，以空心三角形箭頭表示繼承關係。

2.3 Attributes

屬性的選擇遵循「在 Domain Model 中聚焦於資料型別屬性」的原則，避免將複雜概念當作屬性、避免以外鍵（foreign key）代替關聯。各概念類別之屬性整理如下：

Class	Attributes
ComplianceCheck	dateTime（檢查執行時間）、exitCode（結果碼：0=pass, 1=violations, 2=error）
StyleProfile	name（Profile 名稱，用於識別）
ArchitectureConstraint	id（規則識別碼）、description（規則說明文字）
NamingRule	classNamePattern（類別命名的正規表達式 pattern）
DependencyRule	sourcePackage（來源套件 pattern）、targetPackage（目標套件 pattern）、isAllowed（是否允許此依賴方向）
SupertypeRule	targetPackage（適用的套件 pattern）、requiredSupertype（要求繼承或實作的型別名稱）
PackageRule	packagePattern（套件結構的萬用字元 pattern）
Project	rootPath（專案根目錄路徑）
File	filePath（檔案路徑）
ViolationReport	checkedFileCount（已檢查檔案數）、violationCount（違規總數）、suppressedCount（已抑制數量）
Violation	lineNumber（發生行號）、message（違規說明）
Suppression	filePath（被抑制違規所在檔案路徑）、lineNumber（被抑

Class	Attributes
	制違規所在行號）、reason（抑制原因）、timestamp（抑制時間戳記）

3. Design

本章節依序說明 arch-checker 系統的邏輯架構（3.1）、重要使用案例的物件責任分派（3.2）與設計類別模型（3.3）。設計決策以 Domain Model 為基礎。

3.1 Logical Architecture

arch-checker 為單一使用者的 CLI 工具（無 GUI、無資料庫後端）。本節以 UML package diagram 方式呈現系統的組織，並說明各 package 之職責與相依方向。

3.1.1 Package 階層（由上而下）

Layer	Package	Responsibility
Presentation	**com.archchecker.cli**	CLI entry point、指令解析（picocli）、互動式 I/O 與錯誤訊息輸出
Application	**com.archchecker.application**	Use Case Controller：接收系統事件並協調 Domain 完成工作流程；並定義 port 介面（CodeParser、ProfileRepository、SuppressionRepository、Reporter）以隔離 Infrastructure 技術細節
Domain	**com.archchecker.domain.constraint**	ArchitectureConstraint (abstract) 與四個具體規則：NamingRule、DependencyRule、SupertypeRule、PackageRule（多型 validate(files)）
Domain	**com.archchecker.domain.profile**	StyleProfile 設定檔模型，承載專案所選用的 ArchitectureConstraint 集合與相關參數
Domain	**com.archchecker.domain.codebase**	被檢查的程式碼結構：Project（專案）、File（單一原始檔）

Layer	Package	Responsibility
Domain	**com.archchecker.domain.compliance**	合規檢查的流程與結果：ComplianceCheck（檢查紀錄）、Violation、ViolationReport、Suppression
Technical Services	**com.archchecker.infrastructure.profile**	YAML Profile 解析、Schema 驗證（以 SnakeYAML 為基礎）
Technical Services	**com.archchecker.infrastructure.parser**	Java 原始碼解析，將 .java 轉為 AST（以 JavaParser 為基礎）
Technical Services	**com.archchecker.infrastructure.report**	ViolationReport 之 Console / JSON 輸出格式器
Technical Services	**com.archchecker.infrastructure.suppression**	.arch-checker-suppress.yaml 之讀取、寫入與 Suppression 物件的序列化
Foundation	**java.util / java.nio.file / java.time**	集合、檔案 I/O、時間等 JDK 基礎服務

3.1.2 Package 相依關係（dependency）

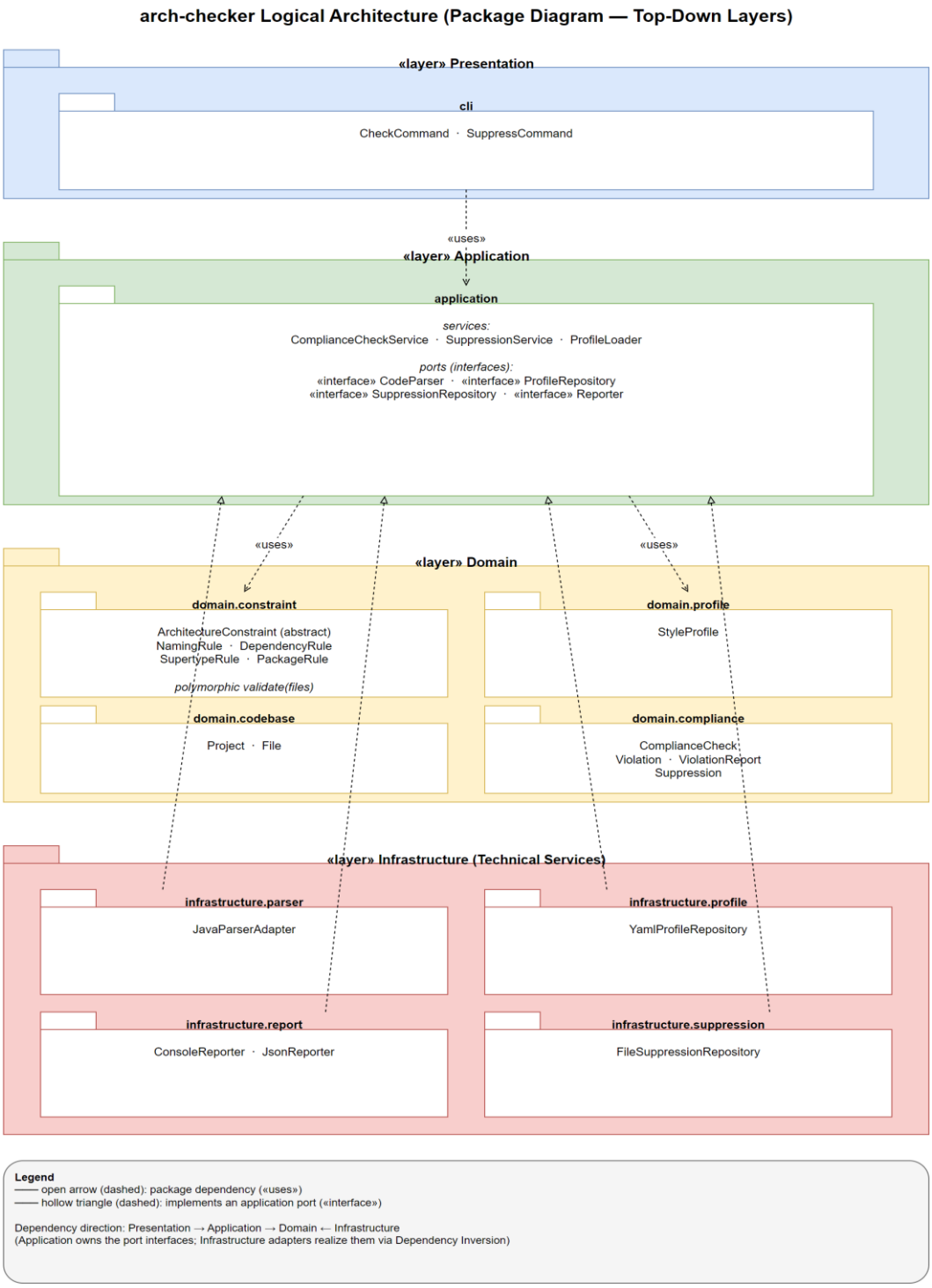
主要相依方向為 Presentation → Application → Domain（虛線箭頭指向 target package）。Infrastructure 層不位於此單向鏈上，而是透過實作 Application 層所定義的 port 介面（CodeParser、ProfileRepository、SuppressionRepository、Reporter，定義位置見 3.1.1）反向相依於 Application，達成 Dependency Inversion；Domain 為核心抽象層，任何其他層皆可依賴它，但它不得相依於任何其他層。

Source Package	Depends On	Nature of Coupling
cli	**application**	CLI 接收指令後，呼叫對應的 use case controller
cli	**domain.compliance**	將 domain 物件（如 ViolationReport）交給 infrastructure.report 輸出
application	**domain.compliance, domain.profile, domain.codebase**	建立 ComplianceCheck、讀取 StyleProfile、收集 Violation

Source Package	Depends On	Nature of Coupling
application	**domain.constraint**	委派 ArchitectureConstraint.validate(files) 進行多型檢查
infrastructure.profile	**application**	實作 application 層的 ProfileRepository port；載入 YAML Profile 並轉為 domain 物件
infrastructure.parser	**application**	實作 application 層的 CodeParser port；解析 .java 為 AST 並轉為 domain 的 File 物件
infrastructure.report	**application**	實作 application 層的 Reporter port；將 ViolationReport 輸出為 Console 或 JSON
infrastructure.suppression	**application**	實作 application 層的 SuppressionRepository port；讀寫 .arch-checker-suppress.yaml 並序列化 Suppression 物件
domain.constraint	**domain.codebase, domain.compliance**	ArchitectureConstraint.validate(files) 接收 File 集合，產生 Violation
infrastructure.*	**domain.compliance, domain.profile**	各 infrastructure 子 package 以 domain 物件為輸入／輸出
(none)	**domain.* (constraint / profile / codebase / compliance)**	Domain 為核心抽象層，不得相依於 cli / application / infrastructure

3.1.3 UML Package Diagram

以下為依 3.1.1、3.1.2 所繪之 UML Package Diagram（如下圖所示）。由上至下為高層至低層；箭頭（虛線）由 source package 指向 target package。



3.2 Use-Case Realizations with GRASP Patterns

本節挑選 3 個使用案例，為每個使用案例繪製 System Sequence Diagram（SSD）以識別系統事件，並為每個系統事件繪製對應的 Sequence Diagram（SD），同時以 GRASP 模式說明責任分派依據。所選三個 use case 分別代表核心檢查、互動修復、違規抑制三條最重要的系統流程。

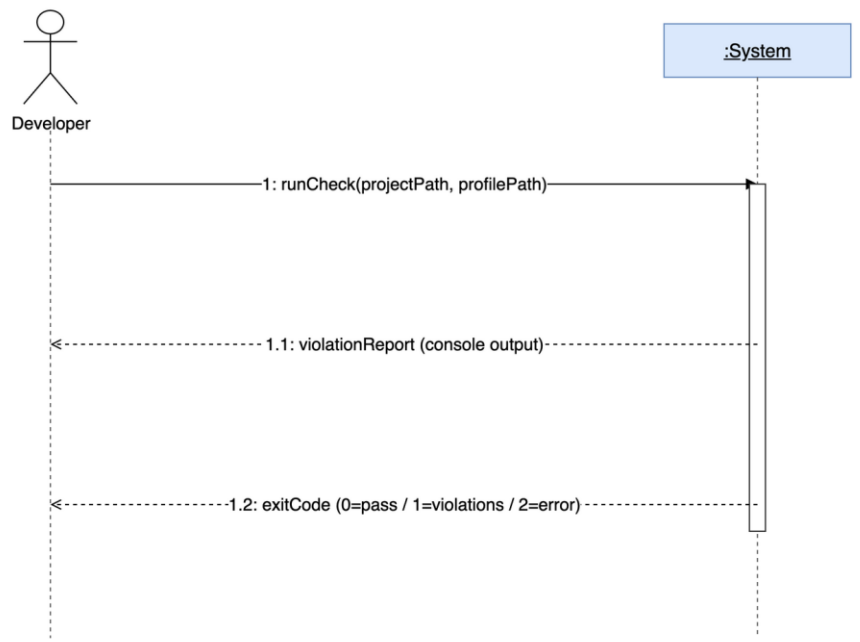
Use Case	Scenario Covered by SSD	Significance
UC-01	Check Architecture Compliance（批次檢查）	系統最核心流程，驗證所有其他 UC 的基礎
UC-03	Check with Interactive Fix（互動修復）	展示系統與使用者的多輪互動，對應 NFR 中的 usability
UC-04	Suppress a Violation（違規抑制）	展示系統與持久化儲存的互動，涉及 Suppression 的生命週期

3.2.1 UC-01: Check Architecture Compliance

(a) System Sequence Diagram

Developer 對 :System 發出單一系統事件 runCheck(projectPath, profilePath)，系統回傳 ViolationReport 並以 exitCode 結束。以下為 SSD 之事件序列：

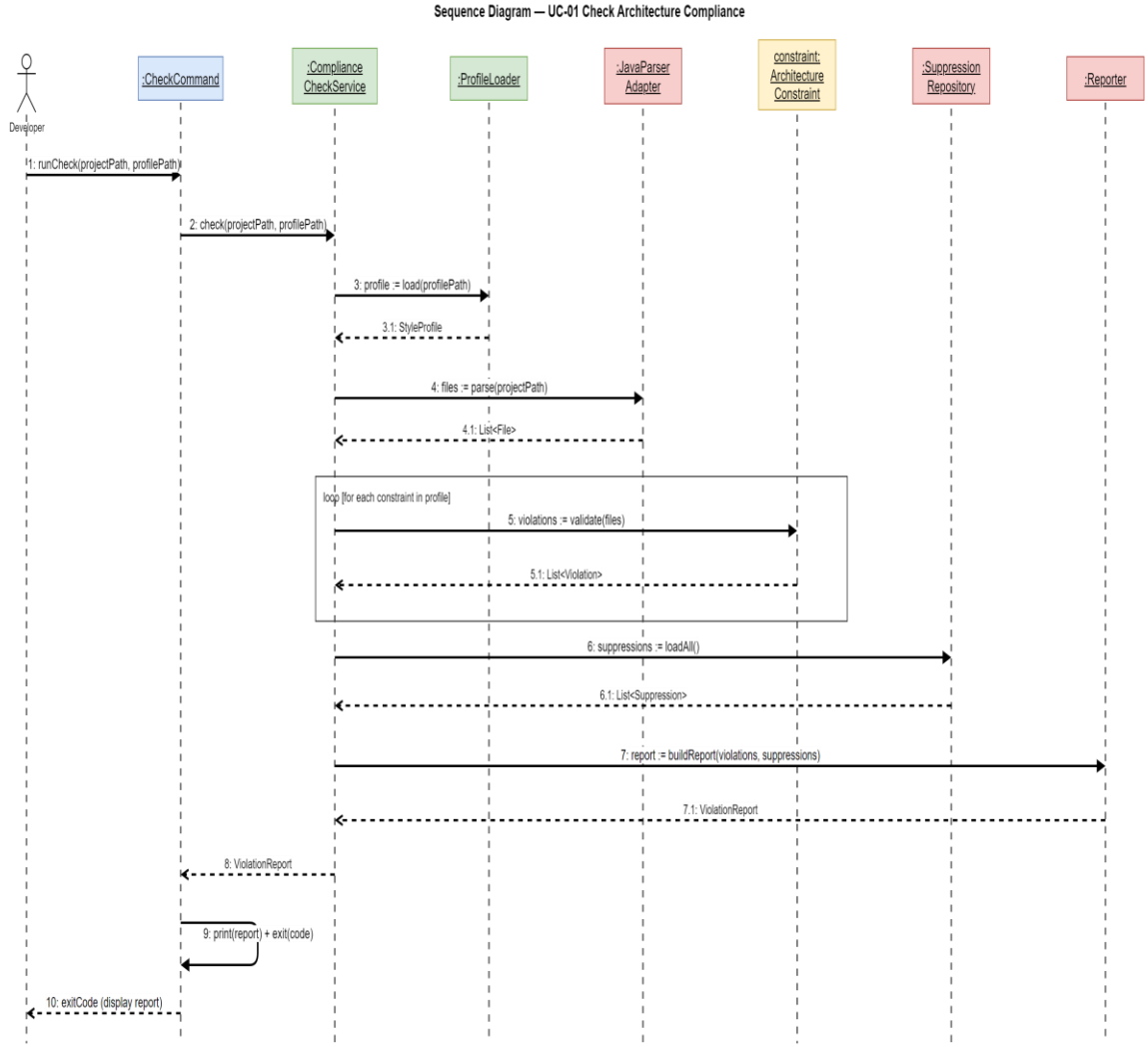
System Sequence Diagram — UC-01 Check Architecture Compliance



事件說明：Developer 於終端機呼叫 arch-checker 進行架構合規檢查。
System 載入 Profile、解析 Java 原始碼、執行規則驗證並產出 ViolationReport，
最後以 console 輸出報告並回傳 exit code 供 CI 判讀。

#	Actor	System Event / Return	Description
1	**Developer**	runCheck(projectPath, profilePath)	啟動一次檢查，指定目標專案與 Profile
2	**:System**	return ViolationReport, exitCode	系統回傳違規報告與對應的 exit code (0/1/2)

(b) Sequence Diagram — System Event: runCheck



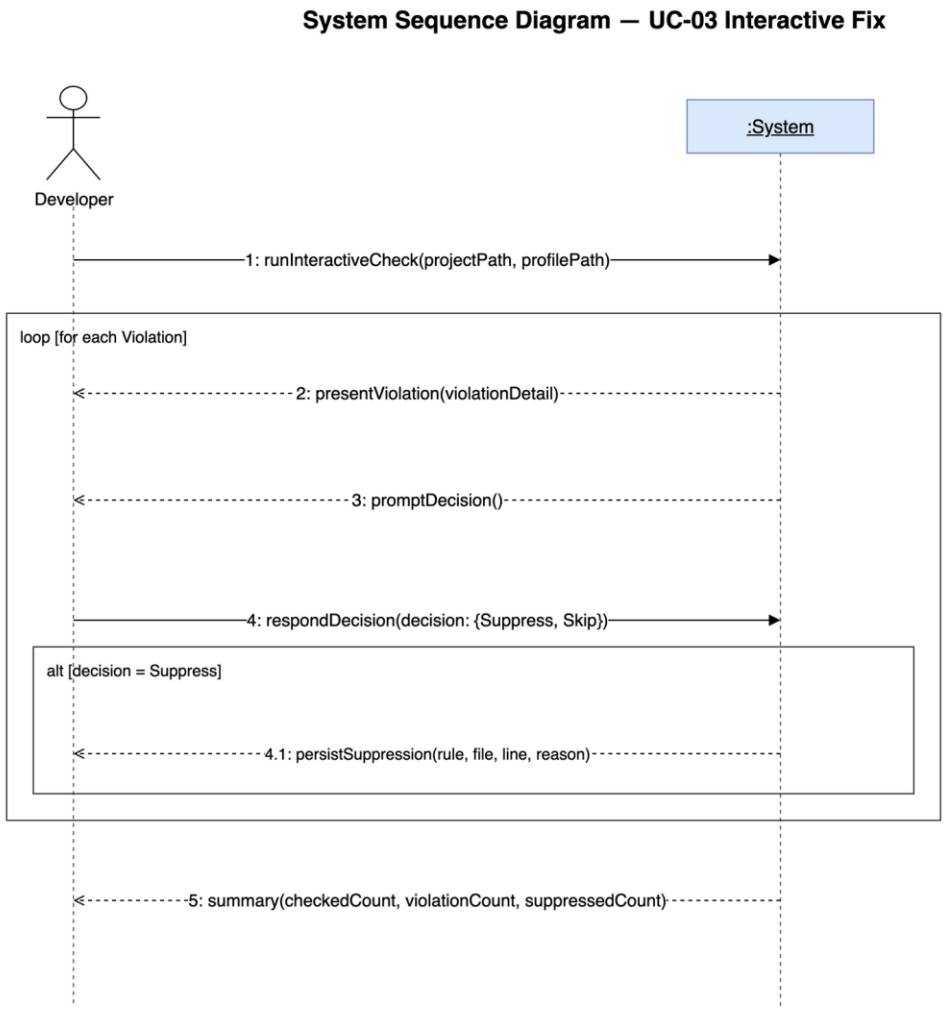
(c) GRASP Rationale Summary (UC-01)

GRASP Pattern	套用對象	Rationale
Controller	:CheckCommand	接收 CLI 系統事件 runCheck(...)，作為 UC-01 use case controller
Pure Fabrication	:ComplianceCheckService、:ProfileLoader、:JavaParserAdapter	為協調 use case 流程或封裝技術細節而建立的非 domain 物件
Polymorphism	constraint:ArchitectureConstraint + 4 Rule 子型別	4 種架構規則檢查邏輯由 validate(files) 動態分派
Information Expert	:SuppressionRepository	擁有抑制清單持久化資訊
Indirection / Low Coupling	:JavaParserAdapter、:SuppressionRepository	以 port/adapter 隔離第三方解析器與儲存細節
Low Coupling	:Reporter	Service 不直接寫 stdout，由 Reporter 處理 Console / JSON 輸出格式

3.2.2 UC-03: Check Architecture Compliance with Interactive Fix

(a) System Sequence Diagram

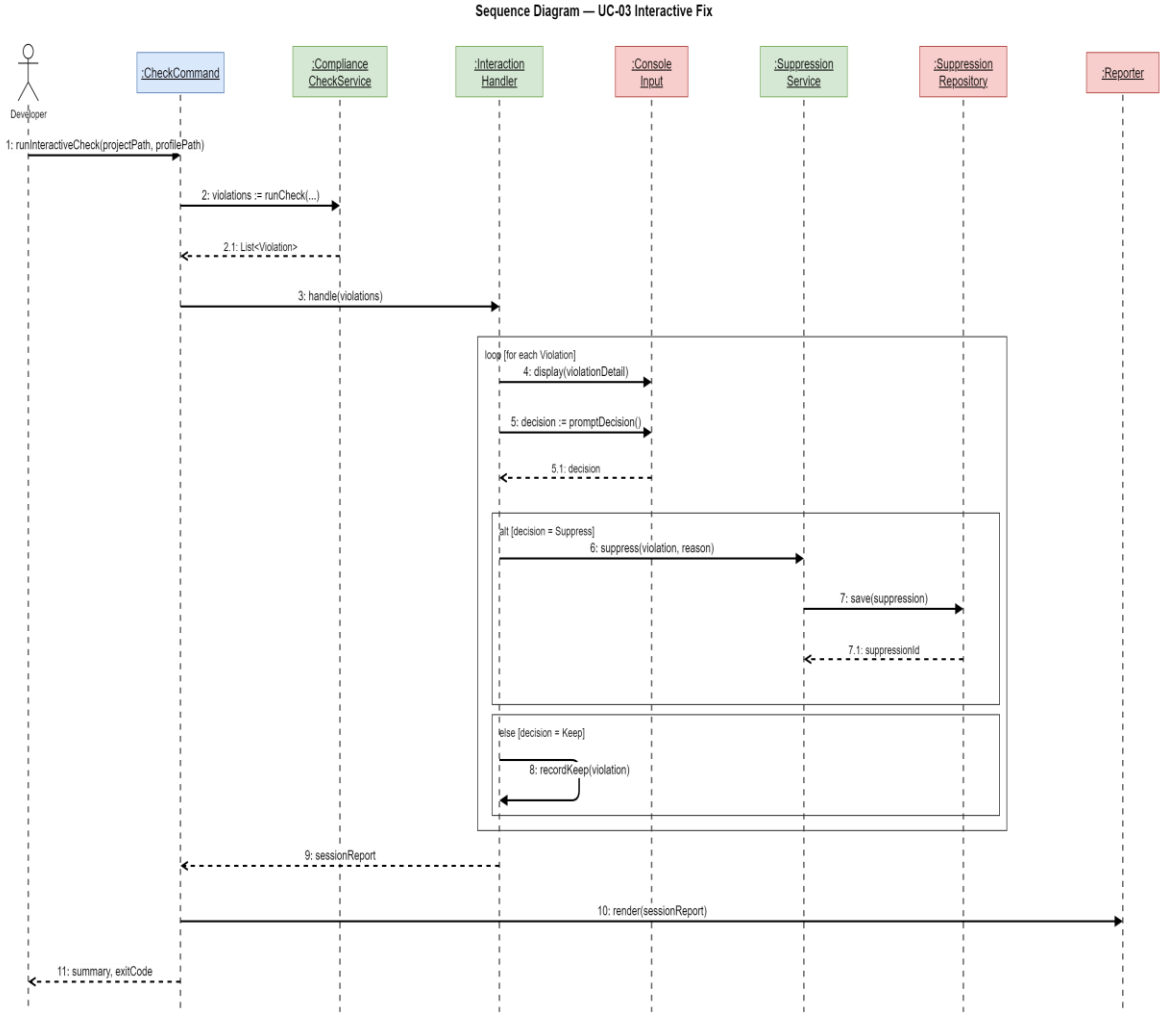
UC-03 於 UC-01 檢查完成後，以 loop 逐條詢問 Developer 對每個 Violation 的處置（Suppress / Keep / Abort）。共有一個入口事件與一個 loop 內的決策事件。



事件說明：UC-03 為 UC-01 的延伸，執行相同檢查但改為互動模式。
系統針對每筆違規逐一呈現細節並等待開發者決策：
若選擇 Suppress，系統立即將抑制記錄持久化；若選擇 Skip，則僅計入報告不抑制。

#	Actor	System Event / Return	Description
1	**Developer**	checkInteractive(projectPath, profilePath)	啟動互動式檢查
2	**:System**	present violation i (details)	系統顯示第 i 筆違規細節
3	**Developer**	decide(i, decision)	decision ∈ {SUPPRESS(reason), KEEP, ABORT}
4	**:System**	return ViolationReport, exitCode	完成 loop 後回傳最終報告

(b) Sequence Diagram — System Event: checkInteractive



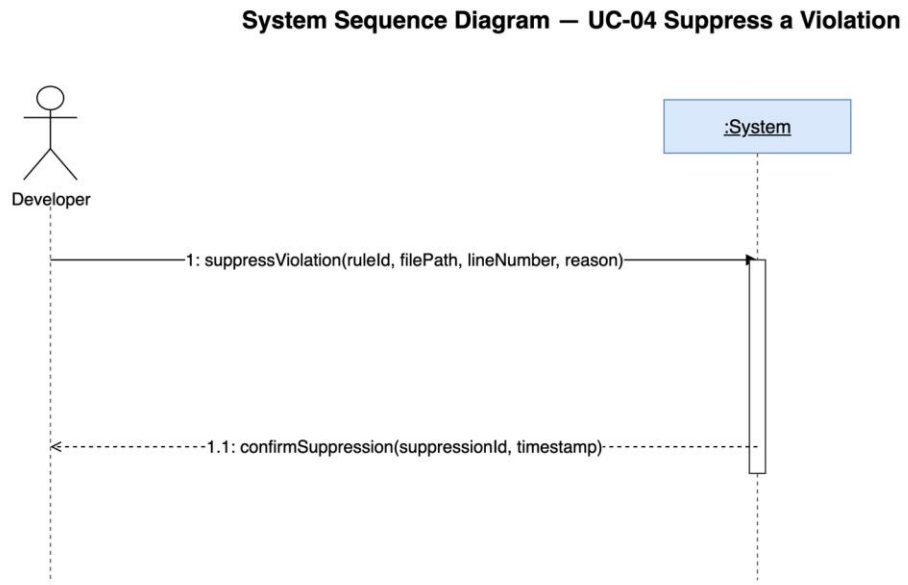
(c) GRASP Rationale Summary (UC-03)

GRASP Pattern	套用對象	Rationale
Controller	:CheckCommand	接收 runInteractiveCheck(...) 作為 UC-03 use case controller
Pure Fabrication	:ComplianceCheckService、:InteractionHandler、:SuppressionService	互動流程協調與 TTY I/O 隔離
High Cohesion	:InteractionHandler、:ConsoleInput	Handler 專責互動編排、ConsoleInput 專責 TTY I/O
Information Expert	:SuppressionRepository	擁有抑制紀錄持久化資訊
Low Coupling	:Reporter	Service 不直接寫 stdout，由 Reporter 處理輸出格式

3.2.3 UC-04: Suppress a Violation

(a) System Sequence Diagram

UC-04 為非互動式的單點抑制：Developer 直接對 :System 發出 suppress 系統事件，系統持久化後回傳確認。

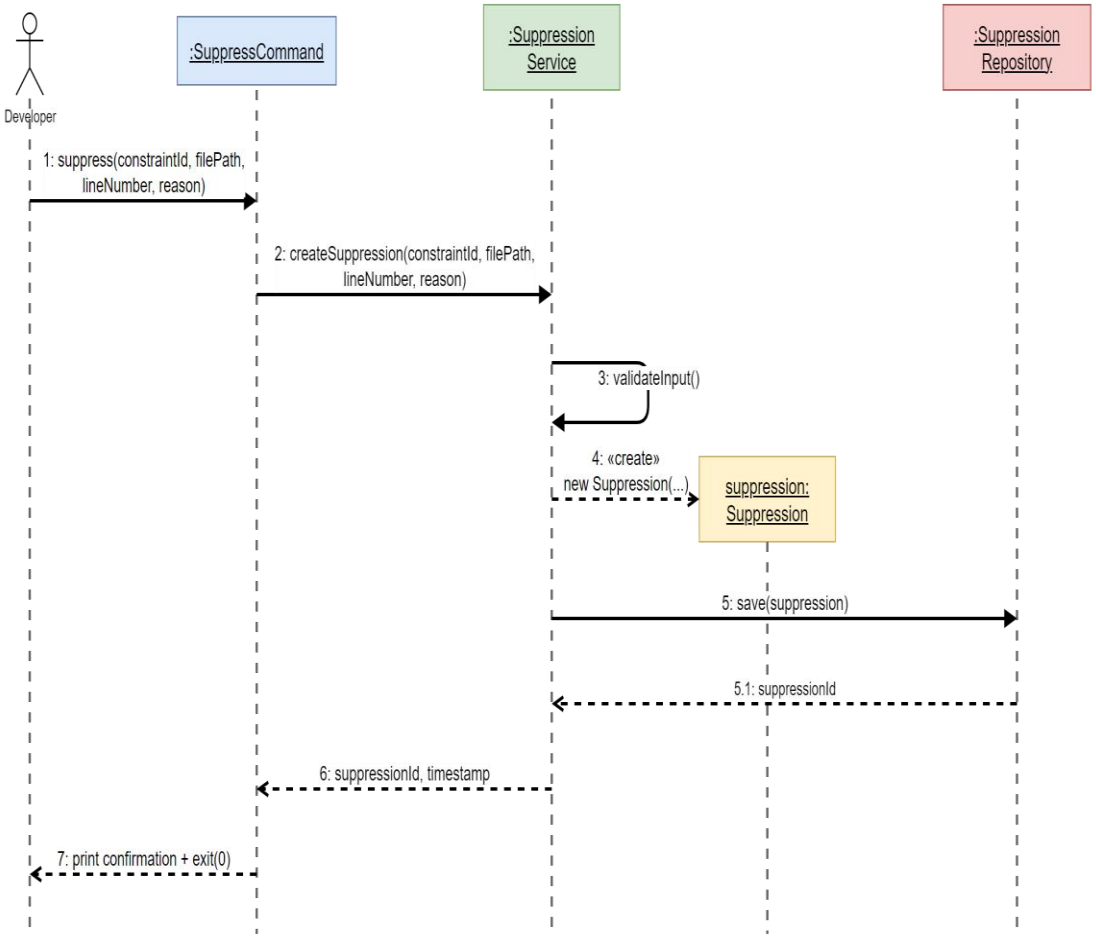


事件說明：Developer 對先前檢查中出現的違規執行非互動式抑制。
System 驗證輸入參數後，將 Suppression 記錄持久化至儲存庫，
並回傳確認訊息（含 suppressionId 與 timestamp）供 Developer 追蹤。

#	Actor	System Event / Return	Description
1	**Developer**	suppress(constraintId, filePath, lineNumber, reason)	要求將指定違規標記為已抑制
2	**:System**	return Suppression (with timestamp) | error	成功回傳 Suppression；失敗則回傳錯誤

(b) Sequence Diagram — System Event: suppress

Sequence Diagram — UC-04 Suppress a Violation



(c) GRASP Rationale Summary (UC-04)

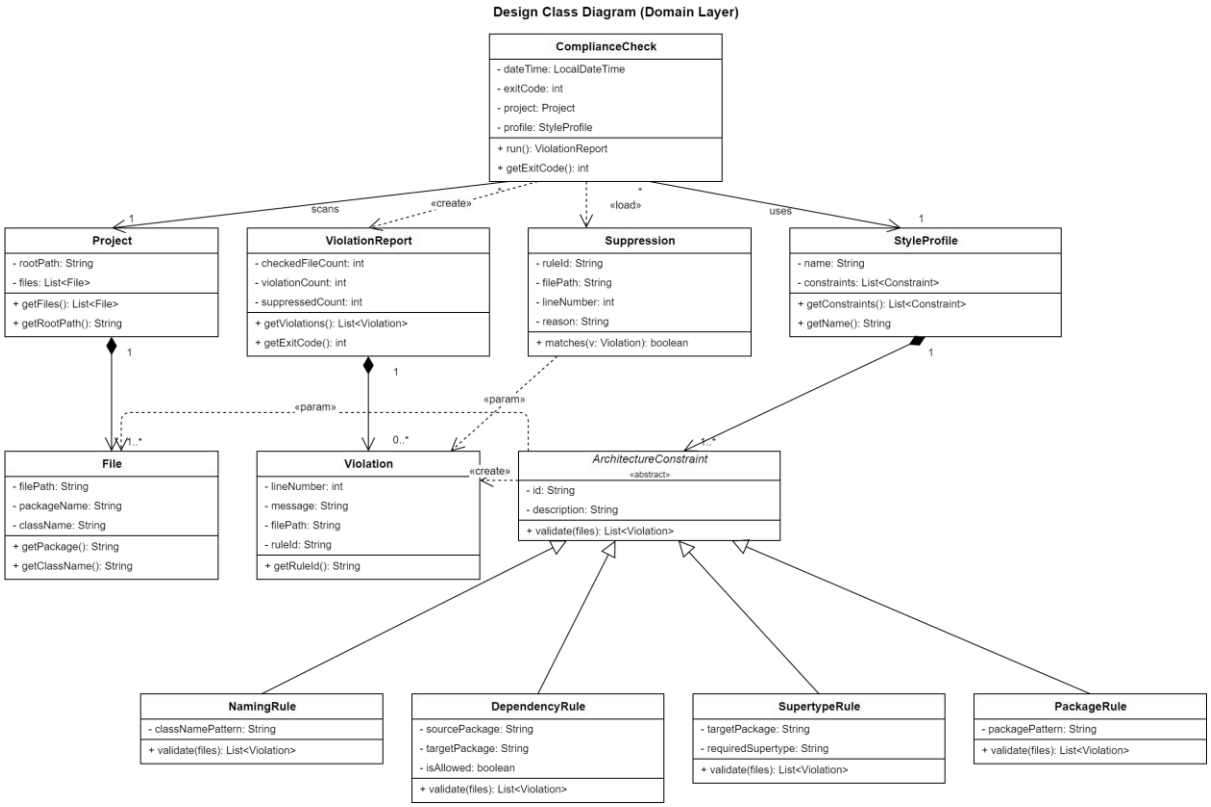
GRASP Pattern	套用對象	Rationale
Controller	:SuppressCommand	接收 suppress(...) 作為 UC-04 use case controller
Pure Fabrication	:SuppressionService	為協調抑制紀錄建立流程而設立
Information Expert	:SuppressionRepository	擁有抑制清單持久化責任
Creator	:SuppressionService → :Suppression	Service 持有建立 Suppression 所需 (constraintId、filePath、lineNumber、reason)

3.3 Design Class Model

本節以 Design Class Diagram（DCD）呈現 arch-checker 之軟體類別設計。

3.3.1 Design Class Diagram (Domain Layer)

下圖為 arch-checker 之 Design Class Diagram，聚焦於 Domain Layer 的 12 個類別與其關聯（generalization、composition、association）。為提升可讀性，類別內的 attribute 與 operation 詳列於下方 3.3.2 Class Specifications 表格；Application Layer（services 與 ports）與 Infrastructure Layer（adapters）已於 3.1 Logical Architecture 之 UML Package Diagram 中呈現。



3.3.2 Class Specifications (Attributes & Operations)

每個類別的 attribute 與 operation 列於下表。可見性符號使用 UML 標準：「-」private、「+」public；「«abstract»」標記抽象類別與抽象 operation。

Class	Attributes	Operations
ComplianceCheck	- dateTime: LocalDateTime - exitCode: int - project: Project - profile: StyleProfile	+ run(): ViolationReport + getExitCode(): int
StyleProfile	- name: String - constraints: List<ArchitectureConstraint>	+ getConstraints(): List<ArchitectureConstraint> + getName(): String
Project	- rootPath: Path - files: List<File>	+ listFiles(): List<File> + getRootPath(): Path
File	- filePath: Path - packageName: String	+ readContent(): String + getFilePath(): Path

Class	Attributes	Operations
ArchitectureConstraint «abstract»	- id: String - description: String	+ validate(files: List<File>): List<Violation> «abstract» + getId(): String
NamingRule	- classNamePattern: String	+ validate(files: List<File>): List<Violation>
DependencyRule	- sourcePackage: String - targetPackage: String - isAllowed: boolean	+ validate(files: List<File>): List<Violation>
SupertypeRule	- targetPackage: String - requiredSupertype: String	+ validate(files: List<File>): List<Violation>
PackageRule	- packagePattern: String	+ validate(files: List<File>): List<Violation>
ViolationReport	- checkedFileCount: int - violationCount: int - suppressedCount: int - violations: List<Violation>	+ addViolation(v: Violation): void + summary(): String
Violation	- lineNumber: int - message: String - file: File - constraint: ArchitectureConstraint	+ describe(): String
Suppression	- filePath: Path - lineNumber: int - reason: String - timestamp: Instant - constraint: ArchitectureConstraint	+ matches(v: Violation): boolean

4. Measurement

Student ID	Name	HW	Date	Duration
113598009	李俊威	HW#1	115/03/03 15:00 ~ 18:00	3H
		HW#2	115/03/12 15:00 ~ 21:00	6H
		HW#3	115/04/01 14:00 ~ 20:00	6H
		HW#4	115/04/17 14:00 ~ 18:00	4H
		HW#4	115/04/22 13:00 ~ 17:00	4H